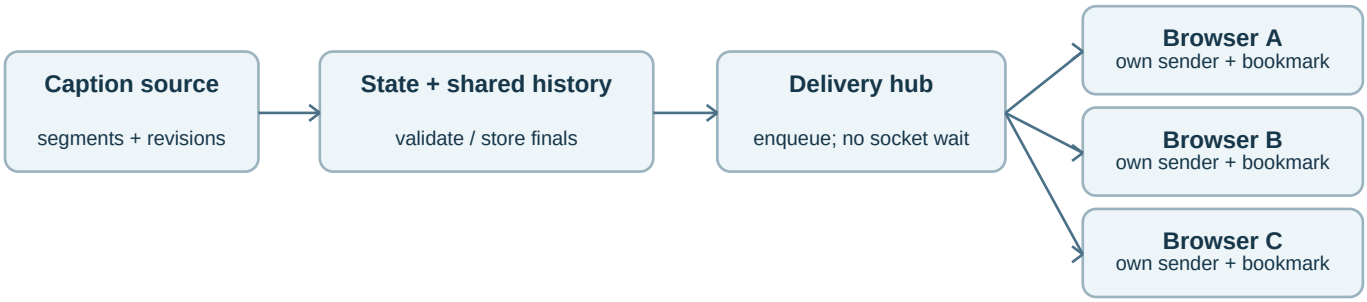
LiveRecall: reliable caption delivery

CS3103 Assignment 4 | Group 15 | Technical report | Validation: 1 November 2026

1. Architecture and compatibility

LiveRecall distributes a changing caption stream and lets students recover recent finalized context. The mandatory system is the supplied deterministic caption source, our WebSocket server, and browser clients. Performance tests use simulated clients inside the same Ubuntu VM; the browser demonstration is separate.



The required entry point is `python solution_server.py --port 8765`. It creates one application and routes the initial SOURCE_HELLO or CLIENT_HELLO to the corresponding session handler. Source updates pass through revision validation and final-history commitment before publication. The delivery hub registers sessions and publishes without waiting for any client socket. Each connection owns its sender task.

Protocol and revision invariants

CAPTION_UPDATE carries session, segment, revision, source_ts, final and text. Positive segment/revision identifiers increase; acceptance requires a newer revision and rejects stale or duplicate state. Once finalized, a segment cannot be overwritten. The source completes a segment before advancing; no fixed revision count is assumed. Extra source fields are ignored, and SOURCE_DONE is not forwarded as a caption.

Finals enter shared history before the state becomes committed and visible to publication. If capacity cannot preserve the declared retention, the update is not silently accepted and lost. Source processing yields to the event loop after publication so buffered input cannot monopolize scheduling. [1,2]

Module ownership of behavior

Module	Responsibility
application.py / source_session.py	Connection routing, accepted-update transaction and source lifecycle
state.py / history.py	Revision/final protection and bounded ordered durable state
delivery_hub.py / client_session.py	Registration, nonblocking publication, client queues and sender policy
client/app.js	In-place revisions, local recall, reconnect checkpoint and HISTORY merge

State, recall and recovery

2. Browser state and local recall

Each segment has one live DOM row. A newer provisional replaces that row; a final changes its style and enters the local finalized map. Older revisions and duplicate finals cannot append a second line. Final history is retained for 70 seconds; the visible live tail is 15 seconds. Recall defaults to 30 seconds, with 15/30/60-second choices using local state without a server round trip. Source timestamps define the age windows.

The browser connects to the hostname serving its page, allowing a Mac browser to use the Ubuntu VM directly. A later recovered final also removes abandoned older provisional rows after a long outage. Browser-state tests cover this case in addition to revision replacement, window selection, checkpoint resume and deduplication.

Atomic attachment and initial history

CLIENT_HELLO reports the latest finalized segment/revision already held by the client and requests up to 60 seconds of context. Under the application lock, attachment selects only missing recent finals, prepares ordered HISTORY, captures the live-history sequence cursor and registers the session. No network await occurs in that transaction. Publication after registration therefore cannot fall into an attachment gap.

The sender transmits initial HISTORY first, then live work. A slow history transfer blocks only that connection. A browser merges history through the same revision/final guards, so reconnect recovery does not duplicate displayed final state. Recently recovered finals may appear in the live tail; older recovered finals remain available in local recall.

Two different progress markers

Marker	Meaning	Lifetime
Browser checkpoint	Last final actually accepted by the browser; reported in CLIENT_HELLO	Survives WebSocket reconnect in the same page
Server next_final bookmark	Next shared-history sequence to send on this connection	Recreated for each attached session

These markers need not differ by exactly one: sequence numbers and segment IDs are distinct, history may be delivered in one initial message, and sending is not proof of application consumption. Reloading a browser page resets its in-memory checkpoint and requests fresh recent context. It does not request the entire lecture.

Finite recovery contract

Server history is time-bounded to 70 seconds, sufficient for requests up to 60 seconds, with hard limits of 20,000 finals and 32 MiB. A connection whose unsent final cursor expires is closed rather than silently jumping over lost state or accumulating an unbounded backlog. An outage beyond retained context cannot recover an unlimited transcript. [1]

Independent bounded delivery

3. Sender policy and resource bounds

A client queue contains only unsent provisionals. Finals remain in shared history and are read through its bookmark. A healthy sender preserves source order: a segment's provisionals precede its final, and a later segment cannot jump ahead of an earlier final. Queue insertion never awaits a socket or full queue.

Bound / signal	Policy
Provisional pending work	At most 256 entries / 1 MiB per connection
Feedback pipeline	At most 8 messages; 64 KiB byte threshold checks after the current message (one-message overshoot possible)
Send / ping timeout	10 seconds for each awaited operation; close only the affected connection
Congestion signals	Queue length/bytes/age, send duration, transport buffer, final lag, and feedback RTT above baseline; severe lag 250 ms; two consecutive mild observations confirm pressure

Under congestion, compaction removes provisionals already superseded by a stored final and keeps the latest queued revision for an active segment. A cadence-aware replacement window applies only to congested provisionals: it is at least 250 ms and at most 1 s. A newer revision restarts the quiet wait, but a 2 s cap prevents perpetual voluntary deferral once the sender can transmit. Network stalls can add further delay. A final removes obsolete work and wakes the sender immediately. Finals are not skipped by compaction.

Why transport feedback is needed

A successful send may only mean bytes entered network buffers. Once sent, obsolete revisions cannot be retracted. Ping/pong checkpoints keep more pending work in the replaceable application queue. Python slow readers eventually pause transport reads when their receive queue fills, delaying pongs. A pong confirms WebSocket transport progress, not browser display or application acceptance. [3,4]

High-RTT optimization

Waiting for a pong after every message limits a healthy link to roughly one message per RTT. The revised policy uses the connection's best measured RTT and an exponentially smoothed accepted-update interval. The default batch is bounded by ceil(2 x baseline RTT / observed interval), a minimum of one and a maximum of eight. A byte cap independently forces a checkpoint. This avoids assuming particular client roles, source revision counts or hidden workloads.

Fast links retain one-message feedback. On longer-RTT links, the small pipeline reduces artificial stop-and-wait delay while the existing pressure policy still suppresses obsolete provisionals. The ping write itself is also timed out, and cancellation cleans up connection work.

Recovery trade-off

High/low thresholds and a quiet interval prevent mode flapping. A large feedback delay can retain coalescing for up to 60 seconds. This conservative hold helped final preservation but can delay restoration of full provisional coverage after a temporary bottleneck. Useful finals continue; a future measured recovery controller should test this trade-off explicitly.

Ubuntu experiments and results

4. Reproducible method

All server, source and simulated clients ran inside the existing VMware Ubuntu 24.04 Mininet lab VM (4 vCPUs, 4 GiB). The main filesystem was extended into already-unused volume-group space; final experiments used the permanent project on disk. The school runner, source, clients and analyser match the supplied starter byte-for-byte. Each run restarts the server. Test parameters, seeds, hashes, raw data, logs, profiles and resource observations are retained in results/final/raw_results.zip.

T1 uses 20 clients for 20 s; T2 uses 100 for 40 s; T3 uses 100 for 60 s with 10% reading at 500 ms/message; T4 uses 100 for 60 s with 20% offline at 15 s for 5 s. T5 uses 100 for 40 s with the supplied 120 ms delay, 30 ms jitter and 3% packet loss on lo. The extra bandwidth experiment applies a separately documented 512 kbit/s TBF cap; it is not a sixth mandatory test.

Test	Final %	Fresh %	Fresh final %	Coverage %	P50 ms	P95 ms	P99 ms
T1	100.00	100.00	100.00	100.00	3.26	5.47	7.91
T2	100.00	100.00	100.00	100.00	10.40	17.78	24.84
T3	100.00	97.07	94.47	94.34	10.38	21.33	5818.82
T4	100.00	100.00	98.42	98.33	9.10	16.67	24.89
T5	100.00	86.75	85.80	62.30	279.13	763.92	1037.30
BAND WIDTH	100.00	31.27	27.00	74.97	569.50	922.92	1201.21

Caption Age is client receive time minus source_ts. Freshness uses the public 500 ms threshold. Completeness counts unique required finals, while update coverage counts source revisions. The table is the median of each run's metric, not percentiles pooled across runs. Per-run completeness remains visible in SUMMARY.md; medians must not hide a weak individual run.

T3 isolation: healthy-client P95 was 17.81 ms with 100.00% freshness; slow-client P95 was 7834.15 ms. Healthy clients retained 20520 live revisions per run (all expected updates). T4 recovery P95 median was 6.97 ms. Recovery timing is interpreted alongside completeness, not as proof on its own.

Observed duplicate/stale/connection-error counts totalled 0. Forty-four Python unit tests and five browser-state tests passed. The wrapper samples server RSS; raw resource observations support short-run boundedness, while hard limits and unit tests establish the declared queue/history caps. Network impairment is reset even when a run fails.

Under slow readers or bandwidth pressure, suppressing obsolete provisionals intentionally lowers overall coverage; unconstrained T1/T2 and the healthy population must still retain full updates. An offline final recovered later cannot meet a 500 ms source-age deadline. TCP loss preserves ordered delivery through retransmission but increases head-of-line and tail delay; a fixed experiment cutoff may leave the last final pending. [3]

Optimization evidence and limitations

5. Before/after optimization comparison

The earlier sender (fb3e54b, one-message feedback) and revised sender were measured with the same T5 command, VM, persistent filesystem and school programs, three runs each. Only the delivery policy changed in the measured server path. Loss is random and three samples do not establish statistical significance; the results support the engineering trade-off rather than a universal guarantee.

Metric	Before median	After median
Final completeness	100.00 %	100.00 %
Fresh final ratio	76.00 %	85.80 %
Revision coverage	50.44 %	62.30 %
P95 caption age	941.41 ms	763.92 ms
P99 caption age	1193.09 ms	1037.30 ms

Bounded RTT-aware batching changed median P95 by -18.9% and revision coverage by +11.86 percentage points. The policy preserves the normal full-update path; measured changes in coverage/freshness reflect the trade-off under impairment. The new byte/message caps keep the pipeline finite. Completeness results should be read exactly as measured, not inferred from freshness.

Failures, limits and AI evaluation

Earlier QEMU-based Ubuntu runs generated a tail final after clients had stopped. An instrumented diagnostic found final 22 about 0.26 s after the client cutoff. A longer receive window was diagnostic only, not a substituted public score. The existing VMware VM showed accurate timers and passed the unchanged basic workload. Final comparisons therefore avoid mixing QEMU, Mac and VMware scores.

A brief 130 ms scheduling spike exposed overly eager coalescing in an earlier normal run. Mild send/feedback delays now require two consecutive observations, while 250 ms severe lag acts immediately; regression tests cover both cases. The initial AI-generated per-message ping design improved slow-reader feedback but added unnecessary serial waits on high-RTT links. We changed it to bounded cadence-aware batching and validated both unconstrained revision coverage and impaired-link behavior. Pongs still cannot identify a UI-only stall if the browser continues processing control frames. Already transmitted work cannot be compacted. Memory and retention caps cannot support an unlimited transcript or arbitrarily long disconnection.

The current public suite is not the hidden evaluator: larger populations, faster/burstier revisions, combined impairment, lower bandwidth and post-overload recovery can expose other limits. The 99% hidden completeness gate is not a promise of a score. The mandatory two-browser video and individual viva remain human deliverables; team contribution details are deferred for team agreement. No optional ASR result is claimed.

Future direction and references

Future work should measure bounded adaptive recovery after temporary overload, application-level receipt feedback for participating browsers, fair scheduling under mixed bottlenecks and deployment capacity. A longer-term classroom system would add session lifecycle, privacy/authentication and optional ASR through the same caption protocol, without changing the mandatory lightweight evaluation path.

[1] NUS CS3103, Assignment 4: LiveRecall, updated 17 October 2026. [2] Supplied public test/protocol guides and unchanged harness; exact configurations and raw evidence in results/final/. [3] RFC 6455, sections 5.5.2-5.5.3: www.rfc-editor.org/rfc/rfc6455.html. [4] websockets documentation, Memory and buffers: websockets.readthedocs.io/en/stable/topics/memory.html. AI/tool use is declared in AI_USAGE.md.